

USER:

All the topics and subtopics in core java for Java full stack developmemnt

ASSISTANT:

For **Java Full Stack Development**, you don't need every obscure part of Java. You need a strong command of **Core Java + modern Java features**, because Spring Boot, Hibernate/JPA, and backend development build heavily on these concepts.

Here is a **complete Core Java roadmap**, arranged from beginner → advanced.

■ Core Java — Complete Topic & Subtopic Roadmap

1. Java Fundamentals

1.1 Introduction to Java

- What is Java?
- Features of Java
- Java vs C/C++
- Java editions: SE, EE/Jakarta EE
- Java applications
- Java ecosystem

1.2 JDK, JRE & JVM

- JDK
- JRE
- JVM
- JDK vs JRE vs JVM
- Java compilation process
- `.java` → `.class` → bytecode
- Java Virtual Machine architecture
- JIT Compiler
- Platform independence

1.3 Installing & Running Java

- Installing JDK
- `JAVA_HOME`
- `PATH`
- `javac`
- `java`
- JShell
- Running Java programs from terminal

- IDEs: IntelliJ IDEA, Eclipse, VS Code

2. Java Syntax & Basics

2.1 First Program

- `main()`
- `System.out.println()`
- Comments
- Identifiers
- Keywords

2.2 Variables

- Variable declaration
- Initialization
- Local variables
- Instance variables
- Static variables
- Constants
- `final`

2.3 Data Types

Primitive

- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `char`
- `boolean`

Non-primitive

- `String`
- `Arrays`
- `Classes`
- `Interfaces`
- `Objects`

2.4 Type Conversion

- Implicit conversion

- Explicit casting
- Widening
- Narrowing
- Type promotion

3. Operators

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Unary operators
- Increment/decrement
- Ternary operator
- Bitwise operators
- Shift operators
- `instanceof`
- Operator precedence
- Short-circuit evaluation

4. Control Flow

4.1 Conditional Statements

- `if`
- `if-else`
- Nested `if`
- `else-if`
- `switch`
- Modern switch
- Switch expressions
- `yield`

4.2 Loops

- `for`
- Enhanced `for`
- `while`
- `do-while`
- Nested loops

4.3 Jump Statements

- `break`

- ``continue``

- ``return``

5. Methods

- Method declaration

- Method calling

- Parameters

- Arguments

- Return values

- ``void``

- Method overloading

- Pass-by-value

- Varargs

- Recursion

- Static methods

- Instance methods

- Method scope

- Method chaining

****Important:**** Java does ****not**** support true pass-by-reference.

6. Object-Oriented Programming ■■■

This is one of the most important sections for Java development.

6.1 Classes & Objects

- Class

- Object

- Object creation

- References

- Object state

- Object behavior

6.2 Constructors

- Default constructor

- Parameterized constructor

- Constructor overloading

- Constructor chaining

- ``this()``

- ``super()``

6.3 Encapsulation

- Data hiding
- Getters
- Setters
- Access control

6.4 Inheritance

- Single inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Why Java doesn't support multiple class inheritance
- `extends`
- `super`

6.5 Polymorphism

- Compile-time polymorphism
- Method overloading
- Runtime polymorphism
- Method overriding
- Dynamic method dispatch

6.6 Abstraction

- Abstract classes
- Abstract methods
- Interfaces
- Interface implementation

6.7 Association Concepts

- Association
- Aggregation
- Composition
- "is-a"
- "has-a"

7. Access Modifiers & Packages

Access Modifiers

- `private`
- default
- `protected`
- `public`

Packages

- Creating packages
- Importing packages
- Static imports
- Package naming conventions
- Subpackages
- Java modules — basic understanding

8. `this`, `super` & `static`

`this`

- Current object reference
- Access instance variables
- Call methods
- Constructor chaining

`super`

- Parent class reference
- Parent constructor
- Parent methods
- Parent variables

`static`

- Static variables
- Static methods
- Static blocks
- Static nested classes
- Static imports

9. Important Java Keywords

You should understand:

`final`

`static`

`this`

`super`

`abstract`

`interface`

`extends`

`implements`

instanceof
synchronized
volatile
transient
native
strictfp
sealed
record

Not all are equally important for daily backend development.

OBPS done

10. Strings ■■■

String Fundamentals

- String creation
- String literals
- String pool
- String immutability
- ``String``
- ``StringBuilder``
- ``StringBuffer``

Important String Methods

- ``length()``
- ``charAt()``
- ``substring()``
- ``equals()``
- ``equalsIgnoreCase()``
- ``compareTo()``
- ``contains()``
- ``startsWith()``
- ``endsWith()``
- ``indexOf()``
- ``replace()``
- ``split()``
- ``trim()``
- ``strip()``
- ``toUpperCase()``
- ``toLowerCase()``

Important Concepts

- ``==`` vs ``.equals()``
- String pool
- Immutability
- String concatenation

11. Arrays

One-Dimensional Arrays

- Declaration
- Initialization
- Traversal
- Searching
- Sorting

Multidimensional Arrays

- 2D arrays
- 3D arrays
- Jagged arrays

Important Concepts

- Array reference
- `length``
- Arrays of objects
- `Arrays`` utility class

12. Wrapper Classes

- Integer
- Long
- Double
- Float
- Character
- Boolean
- Byte
- Short

Concepts

- Boxing
- Unboxing
- Autoboxing
- Auto-unboxing
- `parseInt()``

- `valueOf()`
- Wrapper caching

13. Object Class ■■■

Understand methods inherited from `Object`:

- `toString()`
- `equals()`
- `hashCode()`
- `getClass()`
- `clone()`
- `wait()`
- `notify()`
- `notifyAll()`

Especially:

`equals()` + `hashCode()` contract

This becomes extremely important with Collections, Hibernate/JPA and backend applications.

14. Exception Handling ■■■

Basics

- What is an exception?
- Exception hierarchy
- Errors vs Exceptions

Keywords

- `try`
- `catch`
- `finally`
- `throw`
- `throws`

Types

- Checked exceptions
- Unchecked exceptions
- Runtime exceptions

Important Exceptions

- `NullPointerException`
- `ArithmeticException`

- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`
- `ClassCastException`
- `IllegalArgumentException`
- `IOException`

Advanced

- Multiple catch
- Nested try
- Try-with-resources
- Custom exceptions
- Exception propagation
- Exception chaining
- Best practices

15. Collections Framework ■■■■■

One of the **most important Core Java topics for backend development**.

15.1 Collection Hierarchy

Iterable

↓

Collection

■■■ List

■ ■■■ ArrayList

■ ■■■ LinkedList

■ ■■■ Vector

■ ■■■ Stack

■

■■■ Set

■ ■■■ HashSet

■ ■■■ LinkedHashSet

■ ■■■ SortedSet

■ ■■■ TreeSet

■

■■■ Queue

■■■ PriorityQueue

■■■ Deque

■■■ ArrayDeque

■■■ LinkedList

Map

■■■ HashMap

■■■ LinkedHashMap

■■■ Hashtable

■■■ SortedMap

■ ■■■ TreeMap

■■■ ConcurrentMap

■■■ ConcurrentHashMap

List

- ArrayList
- LinkedList
- Vector
- Stack

Set

- HashSet
- LinkedHashSet
- TreeSet

Queue

- Queue
- PriorityQueue
- Deque
- ArrayDeque

Map

- HashMap
- LinkedHashMap
- TreeMap
- Hashtable
- ConcurrentHashMap

Concepts

- Iterator
- ListIterator
- Comparable
- Comparator
- Sorting collections
- `Collections` class

- `Arrays` class
- Immutable collections

16. Generics ■■■

- Why generics?
- Generic classes
- Generic methods
- Generic interfaces
- Type parameters
- Multiple type parameters
- Bounded types
- Upper bounds
- Lower bounds
- Wildcards
- ``
- ``
- ``
- Type erasure

17. Enums

- Creating enums
- Enum constants
- Fields in enum
- Methods in enum
- Constructors
- `values()`
- `valueOf()`
- Enum with switch

Very useful in backend applications for things like:

ORDER_STATUS

PAYMENT_STATUS

USER_ROLE

18. Date & Time API

Old API

- `Date`
- `Calendar`
- `SimpleDateFormat`

Modern API ■

- LocalDate
- LocalTime
- LocalDateTime
- ZonedDateTime
- Instant
- Duration
- Period
- ZoneId
- DateTimeFormatter

Concepts

- Time zones
- Formatting
- Parsing
- Date arithmetic

19. Functional Programming ■■■■

Modern Java heavily uses functional programming.

Lambda Expressions

- Lambda syntax
- Parameters
- Return values
- Functional interfaces

Functional Interfaces

- Predicate
- Function
- Consumer
- Supplier
- UnaryOperator
- BinaryOperator

Method References

- Static method reference
- Instance method reference
- Constructor reference

Built-in Functional Interfaces

Predicate<T>

Function<T,R>

Consumer<T>

Supplier<T>

20. Stream API ■■■■■

Extremely important for modern Java backend development.

Stream Basics

- What is Stream?
- Stream pipeline
- Source
- Intermediate operations
- Terminal operations

Intermediate Operations

- `filter()`
- `map()`
- `flatMap()`
- `sorted()`
- `distinct()`
- `limit()`
- `skip()`
- `peek()`

Terminal Operations

- `forEach()`
- `collect()`
- `reduce()`
- `count()`
- `min()`
- `max()`
- `findFirst()`
- `findAny()`
- `anyMatch()`
- `allMatch()`
- `noneMatch()`

Collectors

- `toList()`

- ``toSet()``
- ``toMap()``
- ``joining()``
- ``groupingBy()``
- ``partitioningBy()``
- ``counting()``
- ``summarizingInt()``

Advanced

- Parallel streams
- Primitive streams
- Optional with streams

21. Optional ■■■■

- Why Optional?
- Creating Optional
- ``of()``
- ``ofNullable()``
- ``empty()``
- ``isPresent()``
- ``ifPresent()``
- ``orElse()``
- ``orElseGet()``
- ``orElseThrow()``
- ``map()``
- ``flatMap()``
- Optional best practices

Very common in Spring Boot code.

22. I/O — Input & Output

Basic I/O

- `InputStream`
- `OutputStream`
- `Reader`
- `Writer`

File Handling

- `File`
- `FileInputStream`

- `FileOutputStream`
- `FileReader`
- `FileWriter`
- `BufferedReader`
- `BufferedWriter`

Modern File API

- `Path`
- `Paths`
- `Files`

Concepts

- Reading files
- Writing files
- Buffering
- File metadata
- Directory operations
- Try-with-resources

23. Serialization

- Serialization
- Deserialization
- `Serializable`
- `serialVersionUID`
- `transient`
- Object streams

Also understand why modern backend APIs generally use **JSON** rather than Java native serialization for REST communication.

24. Multithreading & Concurrency ■■■■

Thread Basics

- Process vs Thread
- Creating threads
- `Thread`
- `Runnable`
- Thread lifecycle

Thread Methods

- `start()`

- `run()`
- `sleep()`
- `join()`
- `interrupt()`

Synchronization

- Race conditions
- Critical sections
- `synchronized`
- Locks
- Intrinsic monitor

Inter-thread Communication

- `wait()`
- `notify()`
- `notifyAll()`

Concurrency API

- `ExecutorService`
- `Executor`
- `Callable`
- `Future`
- `ScheduledExecutorService`

Synchronization Utilities

- `ReentrantLock`
- `ReadWriteLock`
- `Semaphore`
- `CountDownLatch`
- `CyclicBarrier`
- `Phaser`

Concurrent Collections

- `ConcurrentHashMap`
- `CopyOnWriteArrayList`
- `BlockingQueue`

Atomic Classes

- `AtomicInteger`
- `AtomicLong`
- `AtomicBoolean`
- `AtomicReference`

25. JVM & Memory Management ■■■■

You should understand this at a conceptual level.

JVM Memory

- Heap
- Stack
- Method Area / Metaspace
- Program Counter
- Native Method Stack

Memory Concepts

- Stack vs Heap
- Object allocation
- Garbage collection
- Memory leaks
- Strong references
- Weak references

Garbage Collectors

- Serial GC
- Parallel GC
- G1 GC
- ZGC
- Shenandoah

JVM Concepts

- Class loading
- Class Loader
- Bytecode
- JIT compilation
- Runtime memory

26. Annotations

Built-in Annotations

- `@Override`
- `@Deprecated`
- `@SuppressWarnings`
- `@FunctionalInterface`

- `@SafeVarargs`

Custom Annotations

- Creating annotations
- Annotation targets
- Retention policies
- Runtime annotations
- Reflection + annotations

This becomes **very important** when you start Spring.

27. Reflection API

- Class object
- Getting class information
- Inspecting fields
- Inspecting methods
- Inspecting constructors
- Creating objects dynamically
- Invoking methods
- Access modifiers
- Reflection + annotations

Spring uses reflection extensively.

28. Inner & Nested Classes

- Inner class
- Static nested class
- Local inner class
- Anonymous inner class

29. Interfaces — Advanced

- Default methods
- Static methods
- Private interface methods
- Functional interfaces
- Multiple interface implementation
- Interface inheritance

30. Modern Java Features ■■■■

Java 8

- Lambda
- Functional interfaces
- Stream API
- Optional
- Method references
- Default methods
- Date & Time API

Java 9+

- Modules
- Private interface methods
- Factory methods for collections

Java 10+

- `var`

Java 14+

- Switch expressions

Java 15+

- Text blocks

Java 16+

- Pattern matching for `instanceof`
- Records

Java 17

- Sealed classes

Java 21+

- Pattern matching for switch
- Record patterns
- Virtual threads
- Sequenced collections

For current Java backend development, **Java 17 and Java 21 concepts are particularly worth knowing**, depending on the version used by your project.

31. Records ■■■

Very useful for modern backend applications.

Example:

```
public record User(
```

```
Long id,  
String name,  
String email  
) {}
```

Learn:

- Record syntax
- Canonical constructor
- Compact constructor
- Accessor methods
- Immutability
- Records vs normal classes
- Records as DTOs

32. Sealed Classes

- ``sealed``
- ``permits``
- ``non-sealed``
- Sealed class hierarchy
- Use cases

33. Virtual Threads ■■■

- Platform threads
- Virtual threads
- Why virtual threads?
- Thread-per-request model
- ``Thread.ofVirtual()``
- ``Executors.newVirtualThreadPerTaskExecutor()``
- Virtual threads vs traditional thread pools
- When to use them
- Limitations

34. Networking Basics in Java

- ``InetAddress``
- URL
- URI
- HTTP basics
- `HttpClient`
- HTTP requests
- HTTP responses

- GET
- POST
- Headers
- Status codes
- JSON concept

Modern Java's `java.net.http.HttpClient` is particularly useful.

35. JDBC ■■■■■

Although often taught separately from "Core Java," ****you absolutely need JDBC before serious Spring database work.****

JDBC Architecture

- JDBC
- Driver
- Connection
- Database

Core Interfaces

- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet

Operations

- INSERT
- SELECT
- UPDATE
- DELETE

Advanced

- Prepared statements
- SQL injection prevention
- Transactions
- Commit
- Rollback
- Batch processing
- Connection pooling
- SQLException

36. Design Principles

Before Spring, understand:

SOLID

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Other Principles

- DRY
- KISS
- YAGNI
- Composition over inheritance

These become extremely important when you start writing maintainable Spring Boot applications.

37. Design Patterns ■■■

At least understand:

Creational

- Singleton
- Factory
- Abstract Factory
- Builder
- Prototype

Structural

- Adapter
- Decorator
- Facade
- Proxy

Behavioral

- Strategy
- Observer
- Template Method
- Command

Especially important for Spring

- Singleton
- Factory
- Proxy
- Template Method
- Dependency Injection
- MVC

38. Testing Basics

- Unit testing concepts
- Assertions
- JUnit basics
- Test cases
- Test-driven development basics
- Mocking concepts
- Mockito basics

■ What You Actually Need for Java Full Stack

Don't treat all these topics equally.

■ MUST MASTER

Java Fundamentals

↓

OOP

↓

Classes & Objects

↓

Inheritance

↓

Polymorphism

↓

Abstraction

↓

Interfaces

↓

Exception Handling

↓

Collections

↓
Generics
↓
String
↓
Object class
↓
Lambda
↓
Functional Interfaces
↓
Stream API
↓
Optional
↓
Date & Time
↓
JDBC
↓
Multithreading basics
↓
SOLID

■ SHOULD KNOW WELL

Generics advanced
Enums
I/O
Serialization
Annotations
Reflection
Concurrency
JVM memory
Garbage Collection
Design Patterns
Records
Sealed Classes
Virtual Threads

■ KNOW CONCEPTUALLY

Java Modules
Class Loaders
JVM internals
Advanced GC tuning
Native methods
Advanced reflection
Low-level JVM optimization

■ Where Core Java Fits in Your Full-Stack Roadmap

Your overall path should look like:

JAVA FULL STACK



↓ ↓

FRONTEND BACKEND



HTML → CSS → JavaScript Core Java

↓ ↓

DOM → Events → ES6 JDBC

↓ ↓

React / Angular SQL + DBMS



■ Spring Core



■ Spring Boot



■ REST API / MVC



■ Spring Data JPA



■ Hibernate



■ Spring Security



FULL-STACK PROJECTS



Docker / Git

↓

AWS

****Your immediate priority:**** don't jump into Spring Boot until you're comfortable with ****OOP → Collections → Exceptions → Generics → Lambda → Streams → Optional → JDBC basics****. Those are the Core Java concepts you'll repeatedly encounter in real Spring Boot code.